

SatelliteSimJulia: A Type-Driven, Bare-Array, and End-to-End Differentiable Pipeline for LEO Satellite-Constellation Network Simulation

SatelliteSimJulia Project

Technical report generated and empirically validated on the source tree

Abstract

Low-Earth-orbit (LEO) mega-constellations such as Starlink and Kuiper have turned satellite networking into a large-scale, time-varying graph problem that must be co-designed across orbital mechanics, inter-satellite-link (ISL) physics, topology, routing, traffic, and coverage. We present **SatelliteSimJulia**, an open, modular simulation pipeline written in Julia that stitches the whole chain—Walker constellation generation, orbit propagation, ISL/ground-satellite-link (GSL) evaluation, topology and routing, traffic assignment, and metrics—together through a single bare `Array{Float64,3}` ($N \times T \times 3$) main path, and extends it by multiple dispatch rather than by editing existing code. On top of this the platform adds two capabilities rarely combined in one system: (i) an *end-to-end differentiable* sub-pipeline (analytic J_2 propagation, smooth ISL/coverage relaxations, and an Enzyme/ForwardDiff/Zygote gradient stack) that makes constellation parameters trainable with Adam; and (ii) an *intent anti-corruption layer* that lets a natural-language AI agent drive experiments without ever touching implementation-level names. We describe the architecture and the concrete physics/algorithms implemented, and we validate the platform empirically: a Walker constellation-scale sweep reproduces the sparse-to-connected phase transition (connectivity rising from 3.7% at 48 to 100% at 66 satellites for $P=6$ planes); J_2 -vs-two-body propagation diverges by ~ 822 km over 24 h; a three-way gradient check (forward, reverse, finite difference) agrees to a relative error of 9.2×10^{-6} ; and a differentiable coverage optimizer raises soft global coverage from 38.4% to 77.3% in 100 Adam steps. All results are produced by scripts shipped with the repository.

1 Introduction

The deployment of LEO mega-constellations such as Starlink [1] and Kuiper [2] has changed the scale of satellite networking by orders of magnitude: constella-

tions with hundreds to thousands of cross-linked satellites now form dynamic mesh networks whose topology, routing, and coverage change continuously as satellites move. Designing and evaluating such systems requires simultaneously reasoning about orbital dynamics, optical inter-satellite links, ground access geometry, network topology, routing, and offered traffic—domains that are traditionally handled by separate, loosely-coupled tools.

Two practical problems follow. First, gluing heterogeneous models together usually introduces conversions and abstraction layers at every boundary, which both slow the hot path and make the system hard to extend. Second, constellation design is fundamentally an *optimization* problem (where to place planes and phases to maximize coverage or minimize latency), yet most simulators expose only a forward, non-differentiable evaluation, forcing designers to fall back on grid search or black-box heuristics.

SatelliteSimJulia addresses both problems with a deliberately simple data contract and Julia’s type system [7]:

- **A bare-array main path.** Positions and velocities flow through the entire pipeline as a plain `Array{Float64,3}` of shape $N_{\text{sat}} \times T \times 3$ in ECEF kilometers, with self-describing accessors (`position_at_instant`, `positions_at_last`). There is no per-layer wrapper type on the hot path, which keeps the pipeline allocation-lean, multi-threadable, and—critically—automatically differentiable.
- **Open-closed extension by multiple dispatch.** Propagators, topology strategies, routing algorithms, coverage relaxations, experiments, and AI tools are all **abstract type** hierarchies. Adding a capability means adding a subtype plus one method; existing code is never modified.
- **An end-to-end differentiable sub-pipeline.** An analytic, branch-free J_2 propagator feeds smooth (sigmoid / noisy-OR / LogSumExp) relaxations of coverage and connectivity, so a scalar loss is differentiable with respect to orbital parameters.

Gradients are available through three independent backends (ForwardDiff, Zygote, Enzyme) and drive an Adam optimizer.

- **An intent anti-corruption layer for AI control.** Users and the built-in language-model agent only ever see engineering *intents* (`LowLatencyTopo`, `GlobalCoverage`, `ShortestPath`); a single resolution layer translates these into concrete strategies, so the AI can never depend on, or leak, implementation names.

The remainder of the paper reviews related work (Section 2), details the architecture (Section 3) and the physics/algorithms implemented (Section 4), and reports an empirical validation of the whole stack (Section 5) before discussing limitations (Section 6) and concluding.

2 Related Work

Constellation and space-mission analysis has a mature tool ecosystem, including commercial systems analysis suites and open libraries for orbital mechanics. In the Julia ecosystem, the *SatelliteToolbox* family [6] provides validated implementations of reference-frame transformations, geodetic conversions, analytic Keplerian propagators, and SGP4/SDP4 TLE propagation [5]. *SatelliteSimJulia* follows an “official-library-first” principle and delegates all of these primitives to *SatelliteToolbox* and *SatelliteToolboxSgp4* rather than re-implementing them, focusing its own code on the network-and-optimization layers built on top. The Walker-Delta family of constellations [3] and the astrodynamics relations we rely on follow standard references [4].

Network-level LEO studies typically evaluate ISL topologies (grid/+Grid, Manhattan, motif-based), shortest-path or load-aware routing, and coverage or latency metrics using custom, forward-only simulators. Separately, advances in automatic differentiation [9, 10, 8] have shown that making physical simulators differentiable enables gradient-based design. *SatelliteSimJulia*’s distinguishing feature is that it unifies these threads inside one dispatch-based pipeline: the same bare-array representation that feeds the classical forward metrics also feeds a smooth, differentiable relaxation used for gradient-based constellation optimization, and a language-model agent can orchestrate both through a leak-proof intent layer.

3 System Architecture

The platform is an aggregate package that re-exports a set of single-purpose subpackages arranged along their

Table 1: Subpackage decomposition and dependency direction. Higher rows depend only on lower rows.

Package	Responsibility
Foundation	constants, time grid, frames, entities
Orbit	Walker gen., propagators, ephemeris
Link	ISL/GSL geometry, constraints, capacity
Metrics	coverage, latency, graph analysis
Core	aggregator + catalogs (re-export)
Net	topology strategies, routing, access
Traffic	all-or-nothing traffic assignment
Opt	differentiable propagation + optimization
Lab	intent layer, experiments, AI agent

dependency direction (Table 1). Each layer calls only the public API of the layer below it; the tool layer is the single source of truth, and the orchestration and AI layers are “convenience compositions” that add no new physics.

3.1 The bare-array data contract

All time-indexed state is carried as `positions[i,j,:]`, where i indexes the satellite, j indexes the time slice of a shared `SimulationTimeGrid` (an epoch plus a vector of offsets in seconds), and the last axis holds the ECEF (x, y, z) position in kilometers. Because every layer shares the same time grid, changing the step size once re-aligns the entire pipeline. TLE/SGP4 data joins the same path through a single dispatch on `propagate_to_ecef`, so real ephemerides and synthetic Walker constellations are interchangeable downstream.

3.2 Extension by dispatch

A representative example is topology generation. The abstract contract is `generate_topology(::AbstractTopologyStrategy, T, P)`; a new strategy is a struct plus one method, and it is immediately usable everywhere a strategy is accepted, including inside the intent layer and the AI agent. The same pattern governs propagators (`AbstractPropagator`), routing (`AbstractRoutingAlgorithm`), and coverage relaxations (`AbstractCoverageRelaxation`).

4 Methods

4.1 Constants and time

The platform fixes a small set of physical constants: speed of light $c = 299,792.458$ km/s, WGS-84 equatorial radius $R_{\oplus} = 6378.137$ km, Earth gravitational parameter $\mu = 398,600.4415$ km³/s², Earth rotation rate

$\Omega_{\oplus} = 7.29211 \times 10^{-5}$ rad/s, and the second zonal harmonic $J_2 = 1.0826261732367 \times 10^{-3}$ (EGM96).

4.2 Walker-Delta constellation generation

A Walker-Delta constellation [3] is specified by T total satellites in P planes with phasing factor F ($T = P \cdot S$, $0 \leq F < P$). For plane p and slot s , the right ascension of the ascending node (RAAN) and mean anomaly are

$$\Omega_p = (p-1) \frac{\text{span}}{P}, \quad (1)$$

$$M_{p,s} = \left(360 \frac{s-1}{S} + 360 \frac{(p-1)F}{T} \right) \bmod 360, \quad (2)$$

where $\text{span} = 180^\circ$ for near-polar inclinations ($|i - 90^\circ| < 1^\circ$) and 360° otherwise. The semi-major axis is $a = R_{\oplus} + h$ for altitude h , with a near-circular eccentricity $e = 0.001$.

4.3 Propagation

Keplerian two-body, J_2 , and J_4 propagation are delegated to SatelliteToolbox and produce inertial positions that are then rotated to ECEF via TEME→PEF. TLE inputs use SGP4 (SatelliteToolboxSgp4), with elapsed time measured in minutes from the TLE epoch. Both stacks emit the same bare $N \times T \times 3$ array. For gradient work, the platform additionally provides an *analytic* J_2 propagator (Section 4.7) whose first-order secular rates [4] are

$$\dot{\Omega} = -\frac{3}{2} J_2 n \left(\frac{R_{\oplus}}{p} \right)^2 \cos i, \quad (3)$$

$$\dot{\omega} = \frac{3}{4} J_2 n \left(\frac{R_{\oplus}}{p} \right)^2 (5 \cos^2 i - 1), \quad (4)$$

$$\dot{M} = n + \frac{3}{4} J_2 n \left(\frac{R_{\oplus}}{p} \right)^2 \sqrt{1-e^2} (3 \cos^2 i - 1), \quad (5)$$

with $n = \sqrt{\mu/a^3}$ and semi-latus rectum $p = a(1-e^2)$.

4.4 Link physics

Inter-satellite distance is Euclidean in ECEF and one-way propagation delay is $\tau = d/c$. Line-of-sight between endpoints $\mathbf{a}, \mathbf{b}$ is tested against a spherical Earth of radius $R_{\oplus}$: with $\mathbf{s} = \mathbf{b} - \mathbf{a}$ and $t = \text{clamp}(-\mathbf{a}\mathbf{s}/\|\mathbf{s}\|^2, 0, 1)$, the link is clear iff $\|\mathbf{a} + t\mathbf{s}\| \geq R_{\oplus}$. An ISL is *available* when the distance is within `isl_max_range` (5000 km for the LEO defaults), line of sight is clear, and optional laser-terminal cone/azimuth/duration constraints hold (cone 60° , up to 4 ISLs per satellite). A GSL is available when the range is within `gsl_max_range` (2000 km) and the elevation exceeds `gsl_min_elevation` (25°). Ground-to-satellite elevation is computed in a local NED frame; the RTN frame is used for ISL pointing geometry.

Table 2: Built-in ISL topology strategies. T satellites, P planes, $S = T/P$ per plane.

Strategy	Edge rule	$ E $
Ring	intra-plane ± 1 only	T
Grid+	intra ± 1 + inter same slot	$2T$
Spiral	Grid+ with slot offset	$2T$
Honeycomb	intra + inter if $p+s$ even	$\approx \frac{3T}{2}$
T-shape	3 static + dynamic span-2	var.
Nearest- k	k nearest by 3D distance	$\approx kT/2$
Mesh	complete graph	$\frac{T(T-1)}{2}$

4.5 Topology and routing

Table 2 lists the built-in ISL topology strategies and their edge scaling. Routing on the resulting time-varying graph uses either all-pairs Floyd–Warshall ($O(N^3)$) on propagation-delay weights, Dijkstra shortest paths, equal-cost multipath (ECMP, via k -shortest paths kept within 10^{-9} of the minimum), or a min-load variant whose edge weight is inflated by congestion, $w_i(1 + \text{load}_i/\text{cap}_i)$. End-to-end delay for a ground-to-ground flow sums the source GSL, the ISL path, and the destination GSL delays.

4.6 Metrics and traffic

Coverage is the (weighted) fraction of ground users seen by at least one satellite; latency statistics (mean, p50/p95/p99, diameter) are taken over finite off-diagonal entries of the all-pairs delay matrix; connectivity is the fraction of reachable ordered pairs. Graph-theoretic descriptors include Brandes betweenness centrality [12], PageRank ($d = 0.85$), and the Fiedler value (algebraic connectivity λ_2 of the Laplacian). Traffic is assigned with an all-or-nothing rule per time slice: each active demand is routed on its shortest path if reachable (else dropped), and its rate is accumulated onto the source GSL, destination GSL, and every ISL along the path to produce per-link utilization.

4.7 The differentiable sub-pipeline

To make design objectives differentiable, hard predicates are replaced by smooth surrogates. The core building blocks are a sigmoid soft step and a smooth absolute value,

$$\sigma(kx) = \frac{1}{1 + e^{-kx}}, \quad \text{smooth_abs}(x) = \sqrt{x^2 + \frac{1}{k}}, \quad (6)$$

with default sharpness $k = 20$. Coverage and revisit objectives use four relaxations: (R1) a sigmoid soft elevation threshold $\sigma((el - el_{\min})/\tau)$; (R2) a noisy-OR aggregation across satellites, $1 - \prod_i (1 - c_i)$; (R3) a leaky-integrator revisit gap $g' = (g + \Delta t)(1 - c)$; and (R4) a

LogSumExp soft maximum $\max + \tau \log \sum_i e^{(x_i - \max)/\tau}$. The combined scalar loss is

$$\mathcal{L} = -\overline{\text{cov}} + \lambda \cdot \text{LSE}(\text{revisit gaps}), \quad (7)$$

evaluated over an analytic- J_2 constellation whose positions are a branch-free, dual-number-transparent function of the RAAN/mean-anomaly vector. Gradients are obtained by ForwardDiff [9] (dual numbers), Zygote [10] (reverse mode), or Enzyme [8] (reverse mode); the last drives an Adam optimizer [11] ($\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$) inside the `optimize_coverage` driver, which declares convergence when the relative loss change stays below 10^{-4} for ten consecutive steps. A parallel SGP4 gradient path exposes the seven-element TLE parameter vector $[n_0, e_0, i_0, \Omega_0, \omega_0, M_0, b^*]$ and keeps all string parsing outside the differentiable region.

4.8 Intent layer and AI agent

The Lab layer defines user-facing intents for constellation, topology, routing, traffic, propagator, constraints, and time horizon. A resolution step maps them to concrete implementations given a `ResolutionContext` (e.g. `LowLatencyTopo` resolves to `MeshStrategy` for $T \leq 30$ and to `GridPlusStrategy` otherwise). An `ExperimentConfig` carries the resolved objects into `run_experiment`, which executes the forward pipeline and returns coverage, latency, connectivity, utilization, and a scalar fitness

$$\text{fitness} = (1 - \alpha) \overline{\text{hops}} + \alpha \frac{10 n_{\text{ISL}}}{\max(2T, 1)}, \quad \alpha = 0.5. \quad (8)$$

A ReAct-style [13] language-model agent exposes four tools—`run_simulation`, `scan_parameter`, `compare_constellations`, and `list_available`—all of which speak the intent vocabulary only.

5 Empirical Validation

All experiments below are produced by `scripts/paper_experiments.jl` and `scripts/viz_demo.jl` in the repository, run against the instantiated project on Julia 1.12. Figure 1 and Figure 2 are direct renderings of the forward pipeline.

5.1 Constellation-scale phase transition

We sweep the total satellite count $T \in \{24, 48, 66, 72, 96, 132\}$ at fixed $P = 6$ planes, altitude 780 km, inclination 86.4° , phasing $F = 2$, under the +Grid topology and the LEO ISL constraints, and measure the fraction of *available* ISLs, the network connectivity, and the mean pairwise ISL latency (Table 3, Figure 3). The result is a sharp phase transition: sparse constellations leave inter-plane neighbors beyond the 5000 km ISL range, so connectivity is only a few

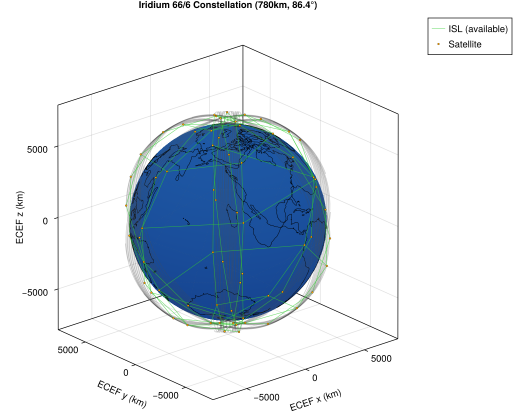


Figure 1: A 3D ECEF snapshot of an Iridium-like Walker 66/6 constellation (altitude 780 km, inclination 86.4°) rendered by the visualization layer: orange markers are satellites, green segments are available +Grid ISLs over the coastline-textured globe.

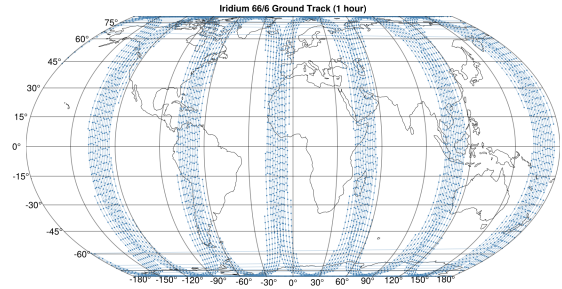


Figure 2: One-hour ground track of the Walker 66/6 constellation over a NaturalEarth coastline base map, illustrating the polar-leaning coverage pattern produced by the 86.4° inclination.

percent; once the constellation is dense enough (here between 48 and 66 satellites) inter-plane links close and the network becomes fully connected, after which the available-ISL fraction saturates near 73% and mean latency slowly *decreases* as denser meshes offer shorter multi-hop paths.

5.2 Propagator fidelity

We propagate the 66/6 constellation for 24 hours (10-minute steps) with both the two-body and J_2 propagators and measure the per-satellite position difference (Figure 4). The divergence grows secularly—as expected from the J_2 -induced RAAN regression and perigee precession—from a mean of 34.2 km at 1 h to 410.9 km at 12 h and 821.5 km (max 822.4 km) at 24 h. This quantifies the modeling error of the fast two-body path and motivates the J_2 (and SGP4) options for

Table 3: Constellation-scale sweep ($P=6$, 780 km, 86.4° , +Grid).

T	ISL avail. (%)	Connectivity (%)	Mean lat. (ms)
24	12.5	2.9	17.3
48	18.8	3.7	21.1
66	68.9	100.0	58.2
72	72.2	100.0	56.0
96	72.9	100.0	53.0
132	72.7	100.0	51.2

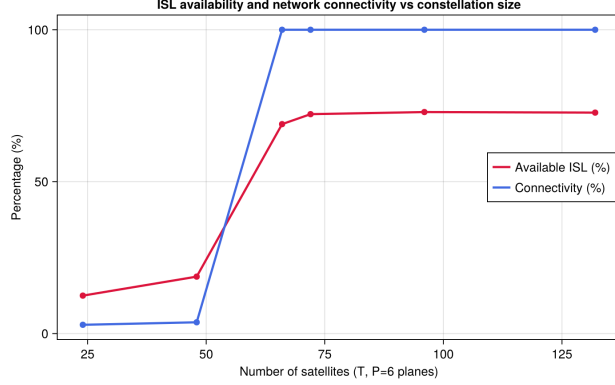


Figure 3: ISL availability and network connectivity versus constellation size. Connectivity jumps from a few percent to 100% between 48 and 66 satellites, marking the point at which cross-plane ISLs fall within range.

longer horizons.

5.3 Gradient correctness

The differentiable claim is only useful if the gradients are correct. Using the end-to-end TLE→SGP4→ISL→soft-route loss ($n = 21$ parameters, loss = 0.7769), we compare gradient norms from ForwardDiff, Zygote reverse mode, and central finite differences (Table 4). Forward and finite-difference gradients agree to a relative error of 9.2×10^{-6} , and reverse mode matches forward mode to 4.1×10^{-15} (machine precision), confirming that the analytic AD path is correct and backend-consistent.

5.4 Differentiable coverage optimization

Finally we exercise the full optimization driver. Starting from a deliberately poor initialization in which all 24 satellites share a single orbital plane ($\Omega = 0$), we minimize the coverage loss of Eq. (7) ($\lambda = 0$, $el_{\min} = 10^\circ$, $\tau = 5^\circ$) over the RAAN/mean-anomaly vector using Enzyme-reverse-mode gradients and Adam (100 steps, learning rate 0.2). Soft global coverage rises rapidly from 38.4% to 77.3%—a gain of 38.9 percent—

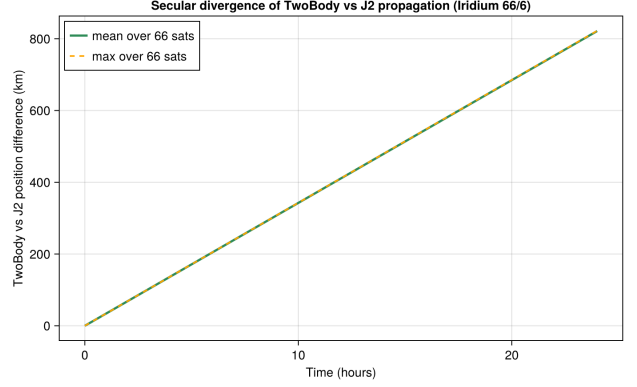


Figure 4: Secular divergence between two-body and J_2 propagation for the 66/6 constellation over 24 h. Mean and maximum are taken across all 66 satellites.

Table 4: Three-way gradient verification of the end-to-end soft-route loss.

Quantity	Value
loss	7.7687×10^{-1}
$\ \nabla\ $ ForwardDiff	9.06966×10^2
$\ \nabla\ $ Zygote (reverse)	9.06966×10^2
$\ \nabla\ $ finite diff.	9.06967×10^2
rel. err. forward vs FD	9.23×10^{-6}
rel. err. reverse vs forward	4.15×10^{-15}

age points—and plateaus after roughly 60 steps (Figure 5). The optimizer autonomously spreads the orbital planes in RAAN, recovering the qualitative structure of a well-phased Walker constellation purely from gradient descent on a coverage objective.

6 Discussion and Limitations

Several deliberate simplifications bound the current results and are worth stating plainly. (i) The experiment runner routes on all-pairs Floyd–Warshall of last-frame ISL delays; the ECMP and min-load intents are recorded but the full per-flow advanced routing is not yet wired into the batch runner. (ii) The coverage optimization in Section 5 uses a single time snapshot and $\lambda = 0$ to isolate the instantaneous coverage objective; the revisit term and multi-epoch trajectories are supported by the loss but not exercised here. (iii) The batch GSL evaluator uses a hard visibility test, while the elevation-dependent capacity models live in the time-series path. (iv) “Soft global coverage” is the smooth surrogate of Eq. (7) rather than a hard geometric coverage percentage, so its absolute value should be read as an optimization signal, not a certified coverage figure. None of these affects the central claims—correct, backend-consistent gradients and a working end-to-end

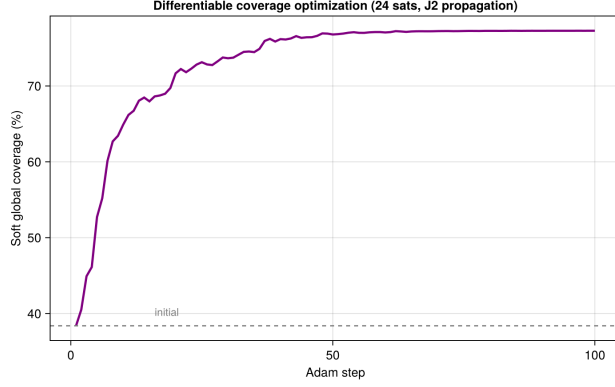


Figure 5: Differentiable coverage optimization. Adam on Enzyme gradients of the smooth coverage loss raises global coverage from a single-plane initialization (38.4%) to 77.3% in 100 steps.

optimization loop—but each is a natural avenue for extension, which the dispatch-based design makes additive rather than invasive.

7 Conclusion

SatelliteSimJulia shows that a LEO constellation network simulator can be both a fast, classical forward pipeline and an end-to-end differentiable model at the same time, by committing to a single bare-array data contract and extending everything by multiple dispatch. The empirical study reproduces a known sparse-to-connected phase transition, quantifies two-body-vs- J_2 modeling error, verifies gradient correctness across three AD backends to near machine precision, and demonstrates gradient-based coverage optimization that improves coverage by nearly 39 points. Together with the intent anti-corruption layer that lets a language-model agent drive experiments safely, these results support the platform’s design thesis: physics fidelity, software extensibility, and differentiable optimization need not be traded off against one another.

References

- [1] SpaceX, “Starlink: Non-geostationary satellite system,” FCC filings, 2016–2020.
- [2] Amazon, “Project Kuiper: Application for a non-geostationary-satellite orbit system,” FCC, 2019.
- [3] J. G. Walker, “Satellite constellations,” *Journal of the British Interplanetary Society*, vol. 37, pp. 559–572, 1984.
- [4] D. A. Vallado, *Fundamentals of Astrodynamics and Applications*, 4th ed., Microcosm Press, 2013.
- [5] F. R. Hoots and R. L. Roehrich, “Models for propagation of NORAD element sets,” Spacetrack Report No. 3, 1980.
- [6] R. Chagas et al., “SatelliteToolbox.jl: A Julia package for satellite analysis,” JuliaSpace, 2019–.
- [7] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, “Julia: A fresh approach to numerical computing,” *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017.
- [8] W. Moses and V. Churavy, “Instead of rewriting foreign code for machine learning, automatically synthesize fast gradients,” *NeurIPS*, 2020.
- [9] J. Revels, M. Lubin, and T. Papamarkou, “Forward-mode automatic differentiation in Julia,” arXiv:1607.07892, 2016.
- [10] M. Innes, “Don’t unroll adjoint: Differentiating SSA-form programs,” arXiv:1810.07951, 2018.
- [11] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *ICLR*, 2015.
- [12] U. Brandes, “A faster algorithm for betweenness centrality,” *Journal of Mathematical Sociology*, vol. 25, no. 2, pp. 163–177, 2001.
- [13] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” *ICLR*, 2023.